Kontrolltöö: kordamine

11IT · Veebirakendused · kontrolltöö 17.11 · 90 minutit · 20 punkti · tunnid 7–9

Kuidas valmistuda

1. Loe proovitöö (lk 2–4) läbi. Kontrolltöö on täpselt samasugune: teooria paberil (10 p) ja kuus TODO-d algfailides (10 p).
2. Tee teooriaosa (lk 3) paberil või vihikus ilma konspektita.
3. Laadi kursuse lehelt alla `proovitoo-algfailid.zip` ja tee kuus TODO-d. Kontrolli serverit käsuga `node kontrolli.mjs`.
4. Võrdle vastuseid ja koodi lahendusega. Kui midagi ei tulnud välja, vaata lahenduse tabelist, mis tunnist see tuli, ja loe see konspekt üle.

Mis selles failis on

Osa	Mis seal on	Lk
Proovitöö	täpselt nagu kontrolltöö: teooria paberil ja kuus TODO-d arvutis	2–4
Lahendus ja kordamine	teooria vastused, kust iga osa tuli, lahenduse kood, sagedased vead	5–8
7. tund: esimene server	konspekt: server, Node.js, Express, GET ja JSON	9–13
8. tund: React ja server	konspekt: fetch, useEffect, laadimine ja veateade	14–20
9. tund: lisamine ja kustutamine	konspekt: POST, DELETE, kontroll ja staatusekoodid	21–27

Lehekülgede numbrid on selle faili omad. Iga osa jalusel on osa enda numbrid.

> **Tule 17.11 tundi arvutiga, kus Node.js ja npm töötavad** (söökla projekt käivitub). Kontrolltöö algfailid saad tunnis.

Proovitöö: küsimused kokale

11IT · Veebirakendused · proovitöö · 17.11 · 90 minutit · kokku 20 punkti

> **See on proovitöö.** Kontrolltöö 17.11 on täpselt samasugune: samad küsimused ja TODO-d, samad punktid ja reeglid. Teised on ainult sõnad, numbrid ja see, mida lehele lisad. Proovitöö algfailid on kursuse lehel `proovitoo-algfailid.zip`.

Kuidas töö käib

Osa	Aeg	Punktid	Kuidas
A. Teooria	umbes 20 min	10	paberil, arvuti on kinni
B. Praktika	ülejäänud aeg	10	arvutis, algfailides on kuus TODO-d

- B-osas on lubatud oma söökla projekt, kursuse lehe konspektid ja tunnijuhendid ning MDN. Keelatud on tehisintellekti tööriistad (ChatGPT, Copilot jms) ja teistega suhtlemine.
- Proovitööd ei pea saatma. Kontrolli serverit käsuga `node kontrolli.mjs` ja võrdle oma vastuseid lahendusega.

Hinne	5	4	3	2	1
Punktid	18–20	15–17	11–14	8–10	0–7

A. Teooria (10 p)

Kirjuta vastused sellele lehele. Arvutit selle osa ajal ei kasuta. Kui oled valmis, anna leht õpetajale ja alusta B-osaga.

Nimi _____ **Kuupäev** _____

1. Mis meetod? (3 p) Kirjuta GET, POST või DELETE.

Tahad näha kõiki küsimusi	
Tahad küsimuse ära kustutada	
Tahad uue küsimuse saata	

2. Mis staatusekood? (4 p) Server töötab nagu B-osa valmis lahendus ja on äsja käivitatud, nii et seal on ainult kaks algset kirjet numbritega 1 ja 2. Iga päring saadetakse sellisele serverile.

`GET /api/kysimused`	
`POST /api/kysimused, keha {"nimi":"Mari","kysimus":"Kas on kala?"}`	
`POST /api/kysimused, keha {"nimi":"","kysimus":"Kas on kala?"}`	
`DELETE /api/kysimused/9`	

3. Loe koodi (1 p) Serveris on marsruut `app.delete('/api/kysimused/:id', ...)` ja tuleb päring `DELETE /api/kysimused/5`. Mis on `req.params.id`? Kirjuta täpselt ja ütle, kas see on arv või tekst.

Väärtus ja tüüp	

4. Leia viga (1 p) Server peaks tühja nime korral vastama 400 ja küsimust mitte lisama. Ometi ilmub tühja nimega küsimus nimekirja. Mis on puudu?

```
if (!nimiOnKorras || !kysimusOnKorras) {
  res.status(400).json({ viga: 'Kirjuta nimi ja küsimus.' });
}
const uus = { id: jargmineKysimusId, nimi, kysimus };
kysimused = [uus, ...kysimused];
```

5. Selgita (1 p) Lisad küsimuse ja taaskäivitad serveri. Küsimus on kadunud. Miks?

B. Praktika (10 p)

Söökla kokk tahab teada, mida õpilased temalt küsida tahavad. Lehel on arvustuste all juba osa „Küsi kokalt“, aga see ei tee veel midagi. Pane see tööle: küsimused tulevad serverist, vormiga saab uue küsimuse saata ja nupuga „Kustuta“ küsimuse ära kustutada.

- Kasuta algfaile: paki lahti, ava kaust `sookla`, tee terminalis `npm install`, siis `cd server`, `npm install` ja `cd ..`.
- Server käib esimeses terminalis (`cd server`, `node index.js`), leht teises (`npm run dev`). Pärast iga muudatust failis `server/index.js` käivita server uuesti.
- Kõik, mida vaja, on juba olemas: andmed, tüüp `Kysimus`, komponent `Kysimused.tsx` koos vormi ja nimekirjaga ning rida `<Kysimused />` failis `App.tsx`. Sinu töö on kuus TODO-kommentaari, mis on koodis kirjas koos juhistega.
- **Iga TODO kohta on eeskuju olemas:** arvustuste kood samas failis või failis `App.tsx`. Kopeeri see ja muuda nimed ära.

TODO	Fail	Mida teha	P
1	`server/index.js`	GET annab kõik kirjed	1
2	`server/index.js`	POST lisab kirje (201) või vastab 400	2
3	`server/index.js`	DELETE kustutab kirje või vastab 404	2
4	`src/Kysimused.tsx`	leht küsib kirjed serverilt	2
5	`src/Kysimused.tsx`	vorm saadab uue kirje serverile	2
6	`src/Kysimused.tsx`	nupp „Kustuta“ kustutab kirje	1

Server peab vastama nii:

Päring	Kõik korras	Midagi valesti
`GET /api/kysimused`	200 ja kõik kirjed	–
`POST /api/kysimused` keha `{ nimi, kysimus }`	201 ja uus kirje koos id-ga	400 ja `{ viga }`, kui nimi või kysimus on puudu või tühi
`DELETE /api/kysimused/:id`	200 ja kustutatud kirje	404 ja `{ viga }`, kui sellist kirjet pole

Punkte saab ka pooliku töö eest: iga TODO hinnatakse eraldi. Tee kõigepealt need, mida kindlalt oskad.

Proovitöö lahendus: küsimused kokale

11IT · Veebirakendused · proovitöö lahendus ja kordamine

Siin on proovitöö vastused ja üks võimalik lahendus koos selgitustega. Sinu kood võib olla teistsugune: oluline on, et server vastaks nagu tabelis ja leht töötaks. Kontrolltöös on täpselt samad küsimused ja TODO-d teise andmetüübiga.

A-osa vastused

Küsimus	Õige vastus	P
1. Tahad näha kõiki küsimusi	GET	1
1. Tahad küsimuse ära kustutada	DELETE	1
1. Tahad uue küsimuse saata	POST	1
2. `GET /api/kysimused`	200	1
2. `POST /api/kysimused, keha {"nimi":"Mari","kysimus":"Kas on kala?"}`	201	1
2. `POST /api/kysimused, keha {"nimi":"","kysimus":"Kas on kala?"}`	400	1
2. `DELETE /api/kysimused/9`	404	1
3. `req.params.id`, päring `DELETE /api/kysimused/5`	`"5"`, tekst (aadressist tuleb alati tekst)	1
4. Leia viga	Pärast res.status(400).json(...) on puudu return;. Ilma selleta läheb funktsioon edasi ja lisab küsimuse ikkagi.	1
5. Lisad küsimuse ja taaskäivitad serveri. Küsimus on kadunud. Miks?	Massiiv on serveri programmi mälus. Kui programm suletakse, kaob ka massiiv ja käivitamisel tehakse see uuesti kahe algse küsimusega. Püsimiseks on vaja andmebaasi või faili.	1

Kordamine: kust see tuli

Mis	Kust õppisime	Mida korrata
Meetodid, staatusekoodid	9. tund, konspekt	GET, POST, DELETE; 200, 201, 400, 404, 500
TODO 1: GET	7. tund, esimene server	`app.get`, `res.json`
TODO 2: POST	9. tund, sammud 1–2	`req.body`, kontroll, `return;`, 400 ja 201
TODO 3: DELETE	9. tund, samm 4	`Number(req.params.id)`, `.find()`, `.filter()`, 404
TODO 4: küsi kirjed	8. tund	`fetch`, `useEffect`, laadimine ja viga
TODO 5: lisa	9. tund, samm 3	fetch POST-iga, päis, `vastus.ok`
TODO 6: kustuta	9. tund, samm 5	fetch DELETE-iga, `.filter()`

Kontrolli serverit ise

Proovitöö algfailides on fail `kontrolli.mjs`. Kui server töötab, tee teises terminalis projekti peakaustas `node kontrolli.mjs`. See saadab serverile samad päringud, millega õpetaja kontrolltööd kontrollib, ja ütleb iga kontrolli kohta „korras“ või „VIGA“.

B-osa lahendus: server/index.js

Kolm TODO-d failis `server/index.js`. Andmed olid juba olemas.

`server/index.js (uus osa)`

```
// ------------------------------------------------------------- Küsimused kokale
let kysimused = [
  { id: 1, nimi: 'Mari', kysimus: 'Kas reedel tuleb ka pannkooke?' },
  { id: 2, nimi: 'Jaan', kysimus: 'Miks on supp alati nii soolane?' },
];
let jargmineKysimusId = 3;

// TODO 1
app.get('/api/kysimused', (req, res) => {
  res.json(kysimused);
});

// TODO 2
app.post('/api/kysimused', (req, res) => {
  const { nimi, kysimus } = req.body;

  const nimiOnKorras = typeof nimi === 'string' && nimi.trim() !== '';
  const kysimusOnKorras = typeof kysimus === 'string' && kysimus.trim() !== '';

  if (!nimiOnKorras || !kysimusOnKorras) {
    res.status(400).json({ viga: 'Kirjuta nimi ja küsimus.' });
    return;
  }

  const uus = { id: jargmineKysimusId, nimi: nimi.trim(), kysimus: kysimus.trim() };
  jargmineKysimusId = jargmineKysimusId + 1;
  kysimused = [uus, ...kysimused];
  res.status(201).json(uus);
});

// TODO 3
app.delete('/api/kysimused/:id', (req, res) => {
  const id = Number(req.params.id);
  const leitud = kysimused.find((x) => x.id === id);

  if (!leitud) {
    res.status(404).json({ viga: 'Sellist küsimust pole.' });
    return;
  }

  kysimused = kysimused.filter((x) => x.id !== id);
  res.json(leitud);
});
```

Kood	Mida see teeb
`res.status(400).json(...);` `return;`	Vasta veaga ja lõpeta, muidu lisataks kirje ikkagi.
`[uus, ...massiiv]`	Uus massiiv: uus kirje ette, vanad järele.
`Number(req.params.id)`	Aadressist tuleb tekst "2", massiivis on arv 2.
`.find(...) / .filter(...)`	find leiab ühe või annab undefined, filter teeb uue massiivi ilma selleta.

B-osa lahendus: src/Kysimused.tsx

TODO 4, 5 ja 6. Ülejäänu oli algfailides valmis.

`src/Kysimused.tsx`

```
import { useEffect, useState } from 'react';
import type { FormEvent } from 'react';
import type { Kysimus } from './types';

export default function Kysimused() {
  const [kysimused, setKysimused] = useState<Kysimus[]>([]);
  const [laeb, setLaeb] = useState(true);
  const [viga, setViga] = useState('');
  const [nimi, setNimi] = useState('');
  const [kysimus, setKysimus] = useState('');

  async function laeKysimused() {
    try {
      const vastus = await fetch('/api/kysimused');
      if (!vastus.ok) {
        throw new Error('Server vastas veaga');
      }
      const andmed: Kysimus[] = await vastus.json();
      setKysimused(andmed);
    } catch {
      setViga('Küsimusi ei saanud kätte. Kas server töötab?');
    }
    setLaeb(false);
  }

  useEffect(() => {
    laeKysimused();
  }, []);

  async function lisa(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    setViga('');

    try {
      const vastus = await fetch('/api/kysimused', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ nimi, kysimus }),
      });
      const andmed = await vastus.json();

      if (!vastus.ok) {
        setViga(andmed.viga);
        return;
      }

      setKysimused((praegused) => [andmed, ...praegused]);
      setNimi('');
      setKysimus('');
    } catch {
      setViga('Serveriga ei saanud ühendust. Kas server töötab?');
    }
  }
```

```
  async function kustuta(id: number) {
    setViga('');

    try {
      const vastus = await fetch(`/api/kysimused/${id}`, { method: 'DELETE' });
      const andmed = await vastus.json();

      if (!vastus.ok) {
        setViga(andmed.viga);
        return;
      }

      setKysimused((praegused) => praegused.filter((x) => x.id !== id));
    } catch {
      setViga('Serveriga ei saanud ühendust. Kas server töötab?');
    }
  }

  return (
    <section>
      <h2>Küsi kokalt</h2>

      <form onSubmit={lisa}>
        <input value={nimi} onChange={(e) => setNimi(e.target.value)} placeholder="Sinu nimi" />
        <input value={kysimus} onChange={(e) => setKysimus(e.target.value)} placeholder="Sinu küsimus" />
        <button type="submit">Küsi</button>
      </form>

      {laeb && <p className="kokku">Laen küsimusi…</p>}
      {viga && <p className="kokku">{viga}</p>}

      <ul>
        {kysimused.map((x) => (
          <li key={x.id} className="kaart">
            <strong>{x.nimi}</strong>
            <p>{x.kysimus}</p>
            <button type="button" onClick={() => kustuta(x.id)}>Kustuta</button>
          </li>
        ))}
      </ul>
    </section>
  );
}
```

Kood	Mida see teeb
`headers: { 'Content-Type': ... }`	Ilma selleta ei loe express.json() keha ja req.body on undefined.
`if (!vastus.ok) { setViga(andmed.viga); ... }`	Näita serveri veateadet ja ära lisa midagi.
`(praegused) => [andmed, ...praegused]`	Pane serveri tagastatud kirje, koos serveri id-ga, ette.
`fetch(`/api/.../${id}`, ...)`	Langetusrõhumärgid: ${id} asemele tuleb kirje number.

Sagedased vead

- Server vastab vana moodi: muutsid faili, aga ei käivitanud serverit uuesti.
- `Cannot destructure ... req.body`: fetch-ist on puudu päis `'Content-Type': 'application/json'`.
- Kirje lisatakse ka pärast 400-t: `res.status(400)...` järel on puudu `return;`.
- DELETE annab alati 404: `req.params.id` on tekst, võrdle `Number(...)`-ga.
- Lehel on kogu aeg „Laen …“: TODO 4 lõpus on puudu `setLaeb(false)`.

Server ja Node.js

11IT · Veebirakendused · seitsmes tund · konspekt

> **Selle tunni suur mõte:** server on lihtsalt programm, mis ootab küsimusi ja vastab neile. Täna kirjutad sellise ise. See tuleb umbes kümme rida pikk.

Mis juhtub, kui avad veebilehe?

Kirjutad brauserisse `tahvel.edu.ee` ja vajutad Enterit. Sekundi pärast on leht ees. Selle sekundi jooksul juhtus kaks asja.

Kõigepealt saatis sinu brauser kuhugi kaugele arvutisse küsimuse: „palun anna mulle see leht“. Seda küsimust nimetatakse **päringuks**. Seal kaugel arvutis töötab programm, mis päringu kätte sai, otsis õige asja üles ja saatis selle tagasi. Seda, mis tagasi tuleb, nimetatakse **vastuseks**.

Programmi, mis päringutele vastab, nimetatakse **serveriks**. Seda, kes küsib (meil brauser), nimetatakse **kliendiks**.

Restoranis	Veebis
Klient istub laua taga ja tellib	**Klient** ehk brauser
Tellimus: „üks hernesupp, palun“	**Päring**
Köök, kus tellimus täidetakse	**Server**
Supitaldrik, mis lauale tuuakse	**Vastus**

Pane tähele, et köök ei tee midagi enne, kui keegi tellib. Samamoodi server. Ta käivitub, ütleb „olen valmis“ ja jääb ootama. Võib-olla tuleb esimene päring kohe, võib-olla tunni pärast. Iga päringu peale teeb ta oma töö ja ootab jälle edasi.

Üks server on sul tegelikult juba olnud. Kui kirjutasid Reacti projektis `npm run dev`, ei tulnud terminali uut `PS …>` rida. See oli sellepärast, et Vite on server ja ta jäi ootama. `Ctrl+C` paneb serveri kinni, nagu köögi uks lukku.

Milleks meil server?

Söökla rakenduses olid arvustused failis `andmed.ts` ja uued arvustused ainult brauseri mälus. Vajutasid F5 ja kõik uued arvustused olid kadunud. Pinginaaber ei näinud neid kunagi, sest need olid ainult sinu arvutis.

Server lahendab selle. Kui arvustused on serveris, küsivad kõik brauserid neid samast kohast. Sinu lisatud arvustust näeb ka pinginaaber. Täna teeme esimese sammu: server, mis annab arvustused välja.

Node.js: JavaScript ilma brauserita

Seni käivitas sinu JavaScripti alati brauser. Serveris pole aga brauserit, pole akent ega nuppe. Seal käivitab JavaScripti programm nimega **Node.js**. Keel on täpselt sama: muutujad, funktsioonid ja massiivid töötavad nagu ennegi.

Kaks erinevust tuleb meelde jätta. Esiteks `console.log` ei kirjuta enam F12 konsooli, vaid **terminali**. Teiseks pole Node’is lehte, nii et `document` ja `querySelector` ei tööta. Neid ei ole serveris vaja.

Node'i programm käivitatakse terminalis nii:

```
node index.js
```

JSON: andmed tekstina

Kujuta ette, et pead sõbrale sõnumis saatma oma tunniplaani. Kirjutad selle tekstina ja sõber saab aru, sest te mõlemad oskate eesti keelt. Server ja brauser on samas olukorras. Nad peavad andmeid saatma, aga mööda internetti liigub ainult tekst. Seega on vaja kokkulepet, kuidas andmed tekstiks kirjutada, et teine pool saaks neist jälle andmed teha.

Selline kokkulepe on **JSON** (loe „džeison", lühend sõnadest *JavaScript Object Notation*). Seda kasutavad peaaegu kõik veebirakendused: ilmateade, Instagram, Bolt. Kui telefonis olev äpp küsib serverilt andmeid, tulevad need väga suure tõenäosusega JSON-ina.

Üks söökla arvustus näeb JSON-ina välja nii:

```
{
  "id": 1,
  "toit": "Kanakaste riisiga",
  "hinne": 5,
  "taimetoit": false
}
```

See on **objekt**: loogelised sulud ja nende sees paarid *nimi: väärtus*, komadega eraldatud. Nimi on alati jutumärkides. Väärtus võib olla:

Väärtus	Näide	Mis see on
tekst	`"Hernesupp"`	alati **jutumärkides**, ülakomad `'…'` ei sobi
arv	`5` või `4.5`	ilma jutumärkideta. `"5"` oleks tekst, mitte arv
tõene või väär	`true`, `false`	jah või ei
tühi	`null`	väärtust pole
massiiv	`[1, 2, 3]`	nimekiri, nurksulgudes
objekt	`{ "nimi": "Mari" }`	objekt objekti sees

Mitu arvustust on **massiiv**, mille sees on objektid. Nurksulud ümber, objektid komadega järjest:

```
[
  { "id": 1, "toit": "Kanakaste riisiga", "hinne": 5 },
  { "id": 2, "toit": "Hernesupp", "hinne": 3 }
]
```

See näeb välja peaaegu nagu JavaScript ja see pole juhus, sest JSON võeti JavaScriptist. Erinevusi on vähe, aga JSON on rangem:

JavaScriptis lubatud	JSON-is
nimed ilma jutumärkideta: `{ toit: "Supp" }`	nimed **alati** jutumärkides: `{ "toit": "Supp" }`
tekst ülakomades: `'Supp'`	tekst **ainult** jutumärkides: `"Supp"`
koma viimase elemendi järel: `[1, 2, 3,]`	viimase järel koma **ei tohi** olla: `[1, 2, 3]`
kommentaarid `// …`	kommentaare ei saa kirjutada

Üks JSON-fail on sul tegelikult juba olemas: `package.json`. Ava see ja vaata. Jutumärgid, koolonid ja komad on täpselt nagu ülal.

> **Hea uudis:** JSON-it ei pea me ise käsitsi kirjutama. Meie serveris on tavaline JavaScripti massiiv ja `res.json(arvustused)` teeb sellest ise JSON-teksti. Järgmises tunnis teeb React sama asja vastupidi: JSON-tekstist saab jälle massiiv. JSON-it on vaja **lugeda** oskada, et brauseris näeksid, mida server saatis.

Pakett ja npm

Serveri saaks kirjutada ka puhta Node'iga, aga see on pikk ja tülikas. Seepärast kasutame **Expressi**. Express on **pakett** ehk kellegi teise kirjutatud kood, mille saad oma projekti ühe käsuga:

```
npm install express
```

`npm` on programm, mis toob pakette internetist. Pärast seda käsku on kaustas kaks uut asja:

- `node_modules` on kaust, kus on Express ja kõik, mida Express ise vajab. Seal on palju kaustu. Ära ehmu ja ära sinna midagi muuda;
- `package.json`-isse tekib rida `"express"`. See on nimekiri: „sellel projektil on vaja Expressi".

Tänu sellele nimekirjale ei pea `node_modules`-it kellelegi saatma. Kes projekti saab, kirjutab `npm install` ja npm toob kõik ise.

Aadress

Meie server töötab sinu enda arvutis. Tema aadress on selline:

```
http://localhost:3000/api/arvustused
```

Osa	Mida tähendab
`localhost`	See arvuti, mille taga sa istud. Server ei ole internetis, vaid sinu juures.
`3000`	**Port** ehk ukse number. Ühes arvutis võib töötada mitu serverit ja igaüks on oma ukse taga. Reacti Vite on uksel 5173, meie server on uksel 3000.
`/api/arvustused`	Mida küsitakse. Ühel serveril võib olla palju aadresse, igaühe taga on midagi muud. `/api` alguses on kokkulepe: siit tulevad andmed.

Esimene server

Terve server mahub kümnesse ritta. Loe see läbi ja siis rida-realt seletus:

`server/index.js`

```
const express = require('express');

const app = express();

app.get('/', (req, res) => {
  res.send('Tere! Söökla server töötab.');
});

app.listen(3000, () => {
  console.log('Server töötab: http://localhost:3000');
});
```

Kood	Mida see teeb
`const express = require('express');`	Toob Expressi paketi sisse. `require` tähendab „võta see pakett kasutusse“.
`const app = express();`	Teeb uue serveri ja paneb selle muutujasse `app`.
`app.get('/', (req, res) => {`	Reegel: kui keegi küsib aadressi `/`, käivita see funktsioon. `get` tähendab, et klient tahab midagi *saada*.
`(req, res)`	Funktsioon saab kaks asja. `req` on päring ehk see, mida küsiti. `res` on vastus ehk sellega saadad midagi tagasi.
`res.send('Tere! ...');`	Saadab vastuseks teksti.
`app.listen(3000, () => {`	Paneb serveri uksel 3000 käima ja ootama.
`console.log('Server töötab...');`	Kirjutab terminali, et server on valmis.

Reeglit „see aadress käivitab selle funktsiooni“ nimetatakse **marsruudiks**. Iga uue aadressi jaoks teed uue marsruudi, kõik täpselt sama kujuga.

Andmed JSON-ina

Teksti saatmine on tore, aga meie tahame saata arvustusi. Paneme serverisse massiivi ja teeme sellele marsruudi:

```
const arvustused = [
  { id: 1, toit: 'Kanakaste riisiga', hinne: 5, kommentaar: 'Parim toit sel nädalal!' },
  { id: 2, toit: 'Hernesupp', hinne: 3, kommentaar: 'Natuke liiga paks.' },
  { id: 3, toit: 'Makaronid hakklihaga', hinne: 4, kommentaar: 'Klassika, alati hea.' },
];

app.get('/api/arvustused', (req, res) => {
  res.json(arvustused);
});
```

Ainus uus asi on `res.json`. See teeb massiivist JSON-i ja saadab selle. Ava brauseris `http://localhost:3000/api/arvustused` ja näed oma arvustusi JSON-ina.

Kaks käsku, mida vajad kogu aeg

Käsk	Mida teeb
`node --watch index.js`	Paneb serveri käima. `--watch` tähendab: kui salvestad faili, käivitab server end ise uuesti. Ilma selleta töötaks edasi vana kood.
`Ctrl+C`	Paneb serveri kinni. Terminal on jälle vaba.

Kui midagi ei tööta

Serveri vead ei tule F12 konsooli, vaid **terminali**, kus server töötab. Vaata alati kõigepealt sinna.

Mida näed	Mida see tähendab	Mida teha
`Cannot find module 'express'`	Selles kaustas pole Expressi.	Kontrolli `pwd`-ga, et oled kaustas `server`. Siis `npm install express`.
`EADDRINUSE` … `3000`	Uks 3000 on juba kinni. Sinu server töötab juba teises terminalis.	Leia see terminal ja vajuta `Ctrl+C`.
`require is not defined`	Sinu `package.json`-is on rida `"type": "module"`.	Kustuta see rida `package.json`-ist.
`Cannot GET /api/…` brauseris	Sellist marsruuti pole.	Võrdle aadressi ja `app.get` rida täht-tähelt.
Brauser ütleb, et lehte ei saa avada	Server ei tööta.	Vaata terminali. Kas server käib või kukkus veaga kokku?
Leht laeb ja laeb	Marsruut ei saatnud vastust.	Kas funktsioonis on `res.send` või `res.json`?
`SyntaxError`	Kirjaviga koodis.	Terminal näitab rea numbrit. Mine sinna.

Sõnastik

Sõna	Tähendus
klient	see, kes küsib. Meil brauser
server	programm, mis ootab päringuid ja vastab neile
päring	küsimus, mille klient serverile saadab
vastus	see, mille server tagasi saadab
Node.js	programm, mis käivitab JavaScripti ilma brauserita
Express	pakett, millega on lihtne serverit teha
pakett	kellegi teise kirjutatud kood, mille saad `npm install`-iga
JSON	kokkulepe, kuidas andmeid tekstina kirjutada. Näeb välja nagu JavaScripti massiivid ja objektid
port	ukse number, mille taga server arvutis ootab
marsruut	reegel: see aadress käivitab selle funktsiooni

React küsib andmed serverilt

11IT · Veebirakendused · 8. tund · konspekt

Selles konspektis selgitan kõik mõisted uuesti läbi, ka need, mida oleme varem kasutanud. See on meelega: mida rohkem kordi sama mõtet kuuled, seda kindlamalt see pähe jääb. Koodi käivitamiseks on vaja Node.js-i, npm-i, VS Code'i ja söökla projekti, mille saad ka kursuse lehelt alla laadida. Lahenda „Proovi ise" kastid, vastused on lõpus.

Pärast seda tundi oskad:
• panna Reacti lehe ja serveri korraga tööle ning selgitada, kes kellega räägib;
• tuua lehe avanemisel andmed serverist ja panna need ekraanile;
• näidata kasutajale laadimisteadet ja veateadet.

Kõige tähtsam on teine punkt. Läbivalt võrdleme kõike restoraniga.

Mis täna muutub?

Meie söökla rakenduse lehel on arvustused. Kust need tulevad? Praegu failist `andmed.ts`, mis asub lehe enda koodi sees. Need on näidisandmed. Justkui toidupilt menüüs: näeb välja nagu päris, aga köögist pole keegi midagi toonud.

Samal ajal on sul olemas ka päris köök ehk server, mis oskab samad arvustused välja anda. Need kaks ei tea teineteisest midagi. Täna ühendame nad. Fail `andmed.ts` läheb prügikasti ja leht küsib andmed serverilt.

Klient, server, päring, vastus

Kõigepealt neli sõna, millel kogu tänane tund seisab.

Restoranis	Veebis	Mis see on
Sina laua taga	**klient**	Programm, mis saadab päringuid. Meil on selleks brauser ja selles töötav Reacti leht.
Tellimus: „üks hernesupp"	**päring**	Küsimus, mille klient saadab.
Köök	**server**	Programm, mis ootab päringuid ja vastab neile.
Taldrik, mis lauale tuuakse	**vastus**	See, mille server tagasi saadab.

Köök ei tee midagi enne, kui keegi tellib. Server samuti: ta käivitub ja jääb ootama. Seepärast ei tule terminali pärast serveri käivitamist uut käsurida. Terminal pole kinni jooksnud, server lihtsalt töötab.

JSON: mis kujul arvustused liiguvad

Interneti kaudu saab saata teksti, pilte, heli ja muid andmeid. Meie rakendus kasutab arvustuste saatmiseks JSON-vormingus teksti. **JSON** (loe „džeison") on kokkulepe, kuidas andmed tekstina kirja panna, et teine pool neist aru saaks. Üks arvustus näeb JSON-ina välja nii:

```
{ "id": 2, "toit": "Hernesupp", "hinne": 3, "kommentaar": "Natuke liiga paks." }
```

Loogelised sulud tähendavad **objekti**: üks asi, millel on nimega väljad. Mitu asja järjest on **massiiv**, selle ümber on nurksulud:

```
[ { "id": 1, "toit": "Kanakaste riisiga" }, { "id": 2, "toit": "Hernesupp" } ]
```

Proovi ise 1. Kas `[ { "id": 1 }, { "id": 2 }, { "id": 3 } ]` on objekt või massiiv? Mitu elementi selles on?

Terminal ja port

Terminal on aken, kuhu kirjutad arvutile käske. Meil on täna neid vaja kaks tükki, sest korraga töötab kaks programmi ja kumbki hõivab oma terminali. Selles projektis on pordid sellised:

Programm	Mida teeb	Käsk	Port
Vite	annab brauserile sinu Reacti lehe	`npm run dev`	5173
Express	annab välja arvustused	`node index.js` kaustas server	3000

Mis on **port**? Mõtle toidusaalile: üks maja, mitu letti, igal letil oma number. Sinu arvuti on maja ja selle aadress on `localhost`, mis tähendab „seesama arvuti“. Port on leti number: selle järgi teatakse, millisele programmile päring läheb. `localhost:3000` tähendab „selles arvutis lett number 3000“. Vite'i vaikimisi port on 5173, aga kui see on juba kinni, võib ta võtta teise. Seepärast ava alati aadress, mille Vite terminali kirjutab.

Kumbki programm jääb tööle, kuni paned ta kinni. Selleks on `Ctrl+C` selles terminalis, kus ta töötab.

Proovi ise 2. Sul on lahti `http://localhost:3000/api/arvustused`. Kumb programm sellele vastab: Vite või Express?

Miks brauser ei lase otse küsida?

Võiks ju arvata, et leht kirjutab lihtsalt `fetch('http://localhost:3000/api/arvustused')` ja kõik. Proovisin. Ei tööta! Brauseri konsoolis on punane veateade sõnaga **CORS**.

Brauser võrdleb lehe ja päringu päritolu: protokolli, hostinime ja porti. Pordid 5173 ja 3000 on erinevad, seega vajab brauser vastuse lugemiseks serveri luba. Meie arendusprojektis kasutame päringu vahendamiseks Vite'i.

Selleks on **proxy** (loe „proksi“) ehk vahendusserver. Restoranis on see teenindaja: sina räägid temaga ja tema viib tellimuse kööki. Brauser küsib Vite'ilt `/api/arvustused`, mis on lehega sama päritoluga. Vite saadab küsimuse serverile edasi ja toob vastuse tagasi.

kumbki server töötab oma terminalis



brauser räägib ainult Vite'iga, Vite räägib serveriga

Päring käib läbi Vite'i, seepärast näeb brauser ainult ühte päritolu.

Pane tähele, et `/api/arvustused` algab kaldkriipsuga ja selles pole serveri nime ega porti. Selline aadress tähendab „sama koht, kust see leht tuli“. Vahendusserveri seadistus läheb faili `vite.config.ts`:

```
server: {
  proxy: {
    '/api': 'http://localhost:3000',
  },
},
```

Inimkeeles: kui aadress algab `/api`-ga, saada päring edasi aadressile `localhost:3000`. Kõik muu annab Vite ise. See seadistus kehtib Vite'i **arendusserveris**. Kui rakendus kunagi päriselt internetti panna, tuleb see lahendada teisiti.

> **Proovi ise 3.** Brauser küsib `/api/menyy`. Kes vastab lõpuks? Aga kui brauser küsib `/favicon.svg`?

React: komponent ja JSX

React on teek, millega ehitatakse veebilehti tükkidest. Üks tükk on **komponent**. Komponent on funktsioon, mille nimi algab suure tähega ja mis tagastab selle, mida ekraanil näidata:

```
export default function App() {
  return <h1>Söökla arvustused</h1>;
}
```

See HTML-i moodi kirjutis on **JSX**. Loogelised sulud JSX-is tähendavad „siin on JavaScript": `{toit}` näitab muutuja `toit` väärtust.

useState: Reacti mälu

Mõtle restorani seinal olevale tahvlile. Kui kirjutad tahvlile uue asja, näevad kõik seda kohe. Reactis on selline tahvel **olek** (ingl *state*) ja see tehakse käsuga `useState`:

```
const [arvustused, setArvustused] = useState<Arvustus[]>([]);
```

Tükk	Tähendus
`arvustused`	mis tahvlil praegu kirjas on
`setArvustused`	funktsioon, millega tahvlile uus asi kirjutada. Pärast seda joonistab React lehe uuesti.
`<Arvustus[]>`	TypeScriptile: siin on massiiv, mille iga element on Arvustus
`([])`	mis on alguses tahvlil: tühi massiiv

Miks mitte tavaline muutuja? Tavalist muutujat muutes React ei tea, et midagi juhtus, ja ekraan jääb samaks.

TypeScript on JavaScript, kuhu saab kirja panna andmete tüübi, näiteks et hinne on arv. Tüüp `Arvustus` asub failis `src/types.ts` ja see jääb alles, kuigi `andmed.ts` kaob.

fetch, async ja await: tellimuse esitamine

Andmeid küsitakse käsuga `fetch`. Päring võtab aega, nagu toit köögist. `fetch()` tagastab kohe Promise'i ehk lubaduse tulevase tulemuse kohta. Nagu teenindaja, kes ütleb „supp on tulemas": supp ise pole veel laual. `await` võimaldab kasutada tulemust pärast seda, kui see on kohale jõudnud.

Meie näites peab `await`-i kasutava funktsiooni ees olema `async`. Funktsioon võib olla `async` ka siis, kui selles pole `await`-i. `await` peatab ainult selle funktsiooni jätkamise, kogu leht ei jää seisma.

```
async function laeArvustused() {
  const vastus = await fetch('/api/arvustused');
  const andmed: Arvustus[] = await vastus.json();
  setArvustused(andmed);
}
```

Kood	Mida see teeb
`async function laeArvustused() {`	Teeme funktsiooni. async lubab selles kasutada sõna await.
`const vastus = await fetch( '/api/arvustused');`	Saadab päringu ja ootab. vastus on nagu kaanega kaetud taldrik: seal on staatus ja sisu, aga kaas on veel peal.
`const andmed: Arvustus[] = await vastus.json();`	Võtab kaane pealt: vastus.json() loeb JSON-vastuse JavaScripti väärtuseks. Ka see võtab aega, sellepärast teine await.
`setArvustused(andmed);`	Kirjutab andmed tahvlile. React joonistab lehe uuesti ja arvustused on ekraanil.

Kirje `: Arvustus[]` aitab TypeScriptil koodi kontrollida, kuid ei kontrolli serveri vastuse tegelikku sisu.

Proovi ise 4. Mis juhtub, kui kirjutad `const vastus = fetch('/api/arvustused');` ilma sõnata await? Kas `vastus` on siis päris vastus?

useEffect: tee see, kui leht avaneb

Funktsioon on olemas, aga keegi pole seda veel välja kutsunud. Millal seda teha? Kohe, kui leht avaneb. Selleks on `useEffect`:

```
useEffect(() => {
  laeArvustused();
}, []);
```

Vaatame seda koodi osade kaupa. `() => { … }` on **noolefunktsioon**, lühidalt kirjutatud funktsioon ilma nimeta. Selle anname Reactile ja ütleme: „käivita see pärast seda, kui leht on ekraanile joonistatud.“

Lõpus olev `[]` on **sõltuvuste massiiv**: nimekiri asjadest, mille muutumisel tuleb funktsioon uuesti käivitada. Tühja sõltuvuste massiiviga `[]` käivitub efekt komponendi lisamisel lehele, mitte iga joonistamise järel.

Miks ei võiks `laeArvustused()` lihtsalt komponenti kirjutada, ilma `useEffect`-ita? Sest iga kord, kui olek muutub, käivitab React kogu komponendi funktsiooni uuesti. Tekiks ring: lae andmed, kirjuta tahvlile, joonista uuesti, lae andmed… Serverile läheks päringuid lõputult.

Ära ehmata: meie projektis on failis `src/main.tsx` sisse lülitatud **StrictMode**, Reacti arendusaegne kontrollrežiim. Selle tõttu toimub arenduse ajal alguses täiendav käivitus. Kui vaatad F12 all vahekaarti Network, näed kahte päringut. See on normaalne.

Mis juhtub, kui midagi läheb valesti?

Server võib olla kinni. Siis peab kasutaja nägema selget teadet, mitte tühja lehte. Vea püüdmiseks on `try` ja `catch`: „proovi seda; kui läks valesti, tee hoopis toda.“

Üks meeldetuletus enne koodi: `true` tähendab „tõene“ ja `false` „väär“. Hüüumärk `!` pöörab selle ümber, `!true` on `false`.

```
  async function laeArvustused() {
    try {
      const vastus = await fetch('/api/arvustused');
      if (!vastus.ok) {
        throw new Error('Server vastas veaga');
      }
      const andmed: Arvustus[] = await vastus.json();
      setArvustused(andmed);
    } catch {
      setViga('Arvustusi ei saanud kätte. Kas server töötab?');
    }
    setLaeb(false);
  }
```

Kood	Mida see teeb
`try { … } catch { … }`	Kui try sees tekib viga, hüppab programm kohe catch-i.
`if (!vastus.ok) {`	vastus.ok on true, kui server vastas õnnestunult (staatus 200–299). !vastus.ok tähendab: kui ei läinud hästi.
`throw new Error('Server vastas veaga');`	Tekitab ise vea, nii et programm hüppab catch-i. fetch ei pea veakoodiga vastust veaks, nii et peame ise kontrollima.
`setViga(...)`	Paneb veateksti olekusse, et seda näidata.
`setLaeb(false);`	Laadimine sai läbi, ükskõik kas õnnestus või mitte. Sellepärast on see rida väljaspool try-d.

Ekraanil näitame teateid nii:

```
{laeb && <p className="kokku">Laen arvustusi…</p>}
{viga && <p className="kokku">{viga}</p>}
```

`&&` tähendab JSX-is: kui vasak pool on tõene, näita paremat poolt. Tühi tekst `''` loetakse vääraks, nii et kui viga pole, ei näidata midagi.

> **Proovi ise 5.** Kui `laeb` on `false` ja `viga` on `''`, mis teade ekraanil on?

Kogu lugu järjest

1. Avad lehe. Vite annab brauserile Reacti koodi.
2. React käivitab App-i. `arvustused` on tühi ja `laeb` on tõene, ekraanil on „Laen arvustusi…“.
3. `useEffect` käivitab `laeArvustused`.
4. `fetch` saadab päringu Vite’ile, Vite saadab selle edasi pordile 3000.
5. Express leiab marsruudi ja vastab JSON-iga. **Marsruut** on reegel „sellele aadressile vasta nii“.
6. `setArvustused` kirjutab andmed tahvlile ja `setLaeb(false)` lõpetab laadimise.
7. React joonistab uuesti. Arvustused on ekraanil.

Kui midagi ei tööta

Mida näed	Võimalik põhjus	Mida kontrollida
„Arvustusi ei saanud kätte…“	Server ei tööta, aadress on vale või vahendusserver pole seadistatud.	Serveri terminal, Vite'i terminal, F12 all Network: mis staatus päringul on. Fail `vite.config.ts`.
Punane CORS-i viga konsoolis	fetch-is on täielik aadress koos pordiga.	Kas fetch-is on ainult `/api/arvustused`.
Konsoolis tuhandeid päringuid	`[]` on puudu või `laeArvustused()` on väljaspool `useEffect`-i.	`useEffect`-i viimane rida.
`map is not a function`	Olekus ei ole massiiv, näiteks puudub `await vastus.json()` või server saatis muud andmed.	Teine `await` ja F12 all Network → Response.
`Cannot find module ...` `andmed`	Fail on kustutatud, aga import on alles.	Kas App-is on veel rida `import { naidisArvustused } ...`.

Sõnastik

Sõna	Tähendus
klient	programm, mis saadab päringuid; meil brauser ja Reacti leht
server	programm, mis ootab päringuid ja vastab neile
päring, vastus	küsimus ja see, mis selle peale tagasi tuleb
marsruut	serveri reegel: sellele aadressile vasta nii
JSON	kokkulepe, kuidas andmeid tekstina kirjutada
port	number, mis ütleb, millisele programmile arvutis päring läheb
localhost	seesama arvuti
päritolu	protokoll, hostinimi ja port kokku
CORS	reeglid, millega server saab lubada brauseril lugeda teise päritoluga vastuseid
proxy, vahendusserver	programm, mis saadab päringud edasi; meil Vite
olek (state)	Reacti mälu; selle muutmisel joonistatakse leht uuesti
useEffect	Reacti käsk: tee see pärast joonistamist
Promise	lubadus tulevase tulemuse kohta
await, async	„kasuta tulemust, kui see on kohal“; ja funktsioon, kus nii tohib teha
try, catch	proovi; kui läks valesti, tee hoopis nii

„Proovi ise“ vastused

1. Massiiv, sest ümber on nurksulud. Selles on kolm elementi.
2. Express, sest port on 3000.
3. `/api/menyy` peale vastab Express, sest aadress algab `/api`-ga. `/favicon.svg` peale vastab Vite ise.
4. Ei. Ilma await-ita on `vastus` alles Promise ehk lubadus, et vastus kunagi tuleb.
5. Ei ühtegi. Mõlemad on väärad.

Arvustuste lisamine ja kustutamine

11IT · Veebirakendused · 9. tund · konspekt

Ka siin selgitan kõik mõisted uuesti, kordamine on meelega. Koodi käivitamiseks on vaja Node.js-i, npm-i, VS Code'i ja söökla projekti, mille saad ka kursuse lehelt alla laadida. Lahenda „Proovi ise" kastid, vastused on lõpus. Veebilehel on **Päringulabor**, serveri simulatsioon, mis töötab brauseris ja kus saad päringuid proovida ilma midagi paigaldamata.

Pärast seda tundi oskad:
• valida õige meetodi: GET, POST või DELETE;
• kirjutada serverile marsruudid, mis lisavad ja kustutavad;
• saata Reactist päringu koos andmetega ja lugeda staatusekoodi.

Kõige tähtsam on kontrollitud arvustuse lisamine. Kustutamine on järgmine etapp. Läbivalt võrdleme kõike restoraniga.

Probleem

Praegu saab meie rakendus andmeid ainult lugeda. Pärast lehe värskendamist kaovad brauseris lisatud arvustused. Serverist saadud algsed arvustused jäävad alles. Miks? Sest uus arvustus jõudis ainult sinu brauseri mällu ja server ei kuulnud sellest midagi.

Täna paneme lehe serverile rääkima: „võta see arvustus vastu" ja „kustuta see ära".

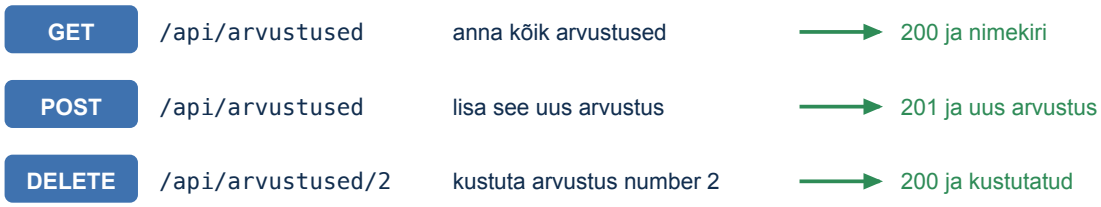
Kes on kes

Restoranis	Veebis	Mis see on
Sina laua taga	**klient**	Programm, mis saadab päringuid: brauser ja Reacti leht.
Tellimus	**päring**	Mida klient serverilt tahab.
Köök	**server**	Programm, mis ootab päringuid ja vastab. Meil Express.
See, mis lauale tuuakse	**vastus**	Mille server tagasi saadab.

Meil töötab korraga kaks programmi, kumbki oma terminalis. **Vite** (`npm run dev`, tavaliselt port 5173) annab brauserile lehe. **Express** (`node index.js` kaustas `server`, port 3000) hoiab arvustusi. **Port** on number, mis ütleb, millisele programmile arvutis päring läheb. Leht küsib aadressilt `/api/...` ja Vite saadab päringu serverile edasi nagu teenindaja, kes viib tellimuse kööki.

Meetod: mida sa tahad teha?

Restoranis võid menüüd vaadata, tellida või tellimuse tühistada. Päringul on samuti lisaks aadressile **meetod**, mis ütleb, mida teha. Sama aadress, eri meetod: eri tegevus.



Sinine on meetod, siis aadress, tähendus ja mida server vastab.

Meetod	Tähendus	Restoranis
`GET`	loe, ära muuda midagi	Vaatad menüüd
`POST`	lisa midagi uut	Annad tellimuse
`DELETE`	kustuta	Tühistad tellimuse

Aadressiribalt saad selles näites teha `GET`-päringu. `POST`-päringu proovimiseks kasutame `fetch()`-i või Päringulaborit.

> **Proovi ise 1.** Mis meetodit kasutaksid: (a) tahad näha kõiki arvustusi, (b) tahad arvustuse number 3 ära võtta, (c) tahad uue arvustuse lisada?

Staatusekood: kuidas läks?

Server vastab alati ka kolmekohalise numbriga, **staatusekoodiga**. Sa oled ühte juba näinud: 404 ehk „ei leitud“. Esimene number näitab vastuse üldist liiki:

Kood	Tähendus	Millal meie server seda kasutab
200	OK, kõik läks hästi	Arvustused anti välja või arvustus kustutati
201	Loodud	Uus arvustus lisati
400	Halb päring: päringus on midagi valesti	Toit on puudu või hinne on 9
404	Ei leitud	Sellise numbriga arvustust pole
500	Serveri ootamatu viga	Seda ei tee me meelega

Rusikareegel: 2-ga algavad tähendavad õnnestumist, 4-ga algavad probleemi päringuga ja 5-ga algavad ootamatut viga serveris.

> **Proovi ise 2.** Saadad arvustuse, mille hinne on 9. Millise staatusekoodiga peaks server vastama? Miks?

Serveri pool: express.json ja req.body

Serveri failis `server/index.js` on **marsruudid**. Marsruut on reegel: „kui tuleb selline päring, tee nii“. Seni oli meil ainult `app.get`. Lisamiseks on `app.post`:

```
app.post('/api/arvustused', (req, res) => {
  // siia tuleb kood
});
```

Funktsioon saab kaks asja. `req` (ingl *request*) on päring: kõik, mida klient saatis. `res` (*response*) on vastus: selle kaudu saadame midagi tagasi. Märk `=>` teeb **noolefunktsiooni**, see on lühidalt kirjutatud funktsioon.

Klient saadab uue arvustuse **päringu kehas** (ingl *body*), meil JSON-tekstina. Aadress ja meetod ütlevad, mida teha, keha on tellimuse sisu. JSON-keha parsimiseks ehk JavaScripti väärtuseks lugemiseks on see rida, mis peab olema pärast `app`-i loomist ja enne marsruute:

```
app.use(express.json());
```

Nüüd saad andmeid lugeda kujul `req.body`. Kui JSON-keha pole parsitud, võib `req.body` olla `undefined` ehk väärtus puudub. Meie praegune kood annaks selle lahtivõtmisel vea.

```
const { toit, hinne, kommentaar } = req.body;
```

Seda kirjutusviisi nimetatakse lahtivõtmiseks. See on lihtsalt lühem kuju kolmest reast: `const toit = req.body.toit;` ja sama teistega.

Ära usalda kedagi: kontrolli andmeid

Vormis on hinne valikmenüüs 1 kuni 5, kas pole siis kõik korras? Ei ole. Päringut saab saata ka ilma sinu vormita ja brauseris saab koodi muuta. Seepärast peab server ise kontrollima.

Mõned märgid enne koodi: `===` tähendab „on täpselt võrdne“, `!==` „ei ole võrdne“, `&&` „ja“, `||` „või“ ning `!` „ei“. `true` on tõene ja `false` väär.

```
  const toitOnKorras = typeof toit === 'string' && toit.trim() !== '';
  const kommentaarOnKorras = typeof kommentaar === 'string' && kommentaar.trim() !== '';
  const hinneOnKorras = Number.isInteger(hinne) && hinne >= 1 && hinne <= 5;

  if (!toitOnKorras || !kommentaarOnKorras || !hinneOnKorras) {
    res.status(400).json({ viga: 'Täida kõik lahtrid, hinne on 1 kuni 5.' });
    return;
  }
```

Kood	Mida see teeb
`typeof toit === 'string'`	toit peab olema tekst.
`toit.trim() !== ''`	trim() eemaldab tühikud algusest ja lõpust. Alles peab jääma midagi.
`Number.isInteger(hinne)`	hinne peab olema täisarv. Tekst "5" ei sobi.
`hinne >= 1 && hinne <= 5`	hinne on lubatud vahemikus.
`if (!toitOnKorras || ...`	Kui vähemalt üks tingimus pole täidetud.
`res.status(400)`	Määrab staatusekoodi.
`.json({ viga: '...' })`	Saadab veateate JSON-ina.
`return;`	Lõpetab funktsiooni kohe, nii et arvustust ei lisata. Ilma selleta läheks kood edasi.

Uue arvustuse tegemine

```
  const uus = { id: jargmineId, toit: toit.trim(), hinne, kommentaar: kommentaar.trim() };
  jargmineId = jargmineId + 1;
  arvustused = [uus, ...arvustused];
  res.status(201).json(uus);
```

Kood	Mida see teeb
`id: jargmineId`	Igal arvustusel peab olema oma number. Numbri paneb server.
`toit: toit.trim()`	Salvestame teksti ilma üleliigsete tühikuteta.
`jargmineId = jargmineId + 1;`	Järgmine saab ühe võrra suurema numbri.
`[uus, ...arvustused]`	Uus massiiv: ees on uus. Kolm punkti koodis laotavad vanad elemendid selle järele.
`res.status(201).json(uus);`	Vastab koodiga 201 ja saadab uue arvustuse koos numbriga tagasi.

Selleks, et muutujatele `arvustused` ja `jargmineId` saaks anda uue väärtuse, peavad need olema kirjutatud sõnaga `let`, mitte `const`.

Kustutamine ja aadressi sees olev number

Kuidas öelda, millist arvustust kustutada? Number pannakse aadressi: `/api/arvustused/2`. Marsruudis tähistab seda koht, mis algab kooloniga. `.find()` otsib massiivist esimese sobiva elemendi. `.filter()` teeb uue massiivi nendest, mis tingimusele vastavad.

```
app.delete('/api/arvustused/:id', (req, res) => {
  const id = Number(req.params.id);
  const leitud = arvustused.find((a) => a.id === id);

  if (!leitud) {
    res.status(404).json({ viga: 'Sellist arvustust pole.' });
    return;
  }

  arvustused = arvustused.filter((a) => a.id !== id);
  res.json(leitud);
});
```

Kood	Mida see teeb
`'/api/arvustused/:id'`	:id tähendab „siia võib tulla mis tahes väärtus, nimetame seda id“.
`req.params.id`	See väärtus. Aadressil /api/arvustused/2 on see tekst "2", sest aadress on alati tekst.
`Number(...)`	Teeb tekstist "2" arvu 2.
`.find((a) => a.id === id)`	Kui ei leia, annab undefined.
`if (!leitud) {`	Kui sellist polnud, vasta 404.
`.filter((a) => a.id !== id)`	Jätab alles kõik, mille id ei ole see. Nii jääb kustutatav välja.

Proovi ise 3. Massiivis on id-d 1, 2 ja 3. Saabub `DELETE /api/arvustused/7`. Millise staatusekoodi saad? Mis juhtub massiiviga?

Kliendi pool: fetch koos meetodiga

Seni kirjutasime `fetch('/api/arvustused')` ja see tegi `GET`-päringu. Teistel meetoditel on vaja teist argumenti: objekt, kus on päringu seaded.

```
const vastus = await fetch('/api/arvustused', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ toit, hinne, kommentaar }),
});
const andmed = await vastus.json();
```

Kood	Mida see teeb
`await`	Kasuta tulemust, kui see on kohal. Funktsiooni ette peab siis kirjutama async.
`method: 'POST',`	Mis meetodiga saata.
`headers: { 'Content-Type': 'application/json' },`	Päringu päised on lisainfo päringu kohta. See ütleb serverile: keha on JSON. Ilma selleta ei parsi express.json() keha.
`JSON.stringify({ ... })`	Teeb objektist JSON-teksti, sest meie server ootab keha JSON-vormingus.
`vastus.json()`	Loeb serveri JSON-vastuse JavaScripti väärtuseks.

Kontrollime ka tulemust. `vastus.ok` on `true`, kui staatus on 200–299. Kui ei, näitame serveri veateadet, mis on kohas `andmed.viga`:

```
if (!vastus.ok) {
  setViga(andmed.viga);
  return;
}
setArvustused((praegused) => [andmed, ...praegused]);
```

`useState` on Reacti mälu ehk olek. Nüüd paneme nimekirja serveri vastuse, sest alles selles on õige `id`. Pane tähele, et `setArvustused` saab nüüd funktsiooni. `praegused` on oleku kõige värskem väärtus. See on vajalik, sest vastus võib tulla alles hiljem: kui vahepeal jõudis mõni teine päring nimekirja muuta, ei kirjutata seda muudatust üle.

Server võib ka üldse mitte vastata. Selleks on kogu päring pandud `try`-plokki, nagu eelmises tunnis.

Kustutamisel pole keha vaja, sest kõik on aadressis:

```
const vastus = await fetch(`/api/arvustused/${id}`, { method: 'DELETE' });
```

Langetusrõhumärgid ehk tagurpidi ülakomad `` ` `` teevad **malliteksti**: `${id}` asemele pannakse muutuja väärtus.

Kuidas nupp saab kustutada?

Kustuta-nupp on komponendis `ArvustuseKaart`, aga nimekiri elab komponendis `App`. Lahendus: `App` annab kaardile funktsiooni `kustuta` **propsina**. Prop on väärtus, mille komponent väljast saab, nagu funktsiooni argument. Kaart ütleb nupuvajutusel: „kutsu see välja minu numbriga“.

```
<button type="button" onClick={() => kustuta(arvustus.id)}>Kustuta</button>
```

Miks `() => kustuta(arvustus.id)`, mitte lihtsalt `kustuta(arvustus.id)`? Teisel juhul kutsutakse funktsioon välja juba joonistamise ajal, ilma nupuvajutuseta, ja see võib arvustusi kustutada. TypeScript võib selle vea juba enne käivitamist märgata.

Aga miks server unustab?

Pane server `Ctrl+C`-ga kinni ja käivita uuesti. Sinu lisatud arvustused on kadunud ja kustutatud on tagasi. Miks? Massiiv `arvustused` elab programmi **mälus**, nagu tekst tahvlil. Programm kinni, tahvel puhas. Käivitamisel kirjutatakse failist uuesti need kolm esimest.

Võrreldes varasemaga on palju paremaks läinud: arvustused jäävad alles lehe värskendamisel. Ühiseid arvustusi näevad kliendid, mis kasutavad sama serverit. Iga õpilase enda arvutis töötaval serveril on eraldi andmed.

Andmete säilimiseks pärast serveri sulgemist tuleb need püsivalt salvestada, näiteks faili või andmebaasi. Edaspidi kasutame selleks andmebaasi.

Kui midagi ei tööta

Mida näed	Võimalik põhjus	Mida kontrollida
Server ei reageeri koodimuudatusele	Serverit pole pärast koodi muutmist uuesti käivitatud.	`Ctrl+C` ja `node index.js`.
Staatus 404, `Cannot POST`	Sellise meetodi ja aadressiga marsruuti pole.	Aadress ja meetod täht-tähelt.
Staatus 500, `Cannot destructure`	`req.body` on `undefined`.	Kas `app.use(express.json());` on enne marsruute ja kas fetch-is on `headers`.
Staatus 400, kuigi kõik on täidetud	Hinne läks tekstina või lahter on ainult tühikuid täis.	Vormi `Number(e.target.value)` ja F12 all Network → Payload.
Arvustused kaovad ilma nupuvajutuseta	`onClick={kustuta(arvustus.id)}`	Kas ees on `() =>`.
„Serveriga ei saanud ühendust“	Server ei tööta või vastus pole JSON.	Serveri terminal ja Network.

Sõnastik

Sõna	Tähendus
meetod	mida päring teha tahab: GET, POST, DELETE
staatusekood	kolmekohaline number vastuses, mis näitab vastuse liiki
marsruut	serveri reegel: selle meetodi ja aadressi peale tee nii
päringu keha (body)	päringuga kaasa saadetud andmed
päringu päis (header)	lisainfo päringu kohta, näiteks mis kujul keha on
parsimine	teksti lugemine andmeteks
req, res	päring ja vastus serveri marsruudis
req.params	aadressi muutuvad osad, näiteks :id
JSON.stringify	teeb objektist JSON-teksti
prop	väärtus, mille komponent väljast saab
andmebaas	programm, mis hoiab andmeid püsivalt, nii et need jäävad alles

„Proovi ise“ vastused

1. (a) GET, (b) DELETE, (c) POST.
2. 400, sest päringus olev hinne ei ole lubatud.
3. 404. Massiiv ei muutu, sest `return` lõpetab funktsiooni enne `filter`-it.